

JAVA SDE-2 INTERVIEW PREPARATION SERIES

FRAMEWORKS, DATA, AND MESSAGING - BOOK 10 OF 12 - STUDY STEP FW-10

Apache Kafka and Spring Kafka

Logs, Partitions, Consumer Groups, Delivery Semantics, and Java Integration

BY

Vinay Reddy Kalluri

One coherent guide from Kafka fundamentals to Spring Kafka configuration, consumers, producers, retries, tests, and operations.

KAFKA | SPRING KAFKA | PARTITIONS | OFFSETS | DELIVERY

JAVA 21 | INTERVIEW CORE | SDE-2 FOLLOW-UPS | PRINTABLE EDITION

Series edition - Updated 2026-08-02

Open educational interview-preparation edition

Start Here - Your Route Through This Volume

60-second entry rule: If two or more recognition signals below expose a gap, begin with Chapter 1. If the prerequisites are weak, step back to FW 09 - Persistence, SQL, and Caching. If you can already explain every outcome without notes, jump to Part II and prove it through the practice ladder.

Keep Apache Kafka semantics and Spring Kafka integration together while teaching the platform model before framework abstractions.

Readiness map

Decision	Evidence
START HERE	Services communicate asynchronously; Ordering, replay, or consumer scaling matters; Retries and delivery semantics can duplicate effects.
PREPARE FIRST	FW-04 Spring Boot; Database transaction basics.
MOVE ON WHEN	Reason from partitioned logs, keys, acknowledgements, and offsets; Design idempotent delivery, schema evolution, retry, DLT, and outbox or inbox flows; Configure Spring Kafka and operate lag, rebalances, security, and incidents.

Choose the route that fits today

- 1 **Learning:** read in order, type the representative Java examples, and complete each dry run.
- 2 **Revision or rehearsal:** start at the first decision map, invariant, or failure mode you cannot explain; if none expose a gap, go directly to Part II and prove readiness without notes.

Frameworks Segment Position

FW 09 - Persistence, SQL, and Caching	YOU ARE HERE FW 10 - Apache Kafka and Spring Kafka	FW 11 - Spring Ecosystem Extensions
---------------------------------------	--	-------------------------------------

A note from Vinay

Kafka guarantees are built from several contracts—partition order, producer acknowledgement, consumer offsets, and application side effects. Name each boundary before saying 'exactly once.'

Contents

This contents page covers only the current focused PDF. Section-level navigation is available through PDF bookmarks.

CONTENTS NAVIGATION	
Start Here - Your Route Through This Volume	2
Part I - Core Learning	4
Chapter 1 - Apache Kafka and Spring Kafka	4
Chapter 2 - Broker Log, Partitions, Replication, and Storage Flow	7
Chapter 3 - Producers: Keys, Batching, Acknowledgements, Idempotence, and Transactions	10
Chapter 4 - Consumers: Groups, Offsets, Polling, Rebalances, and Delivery	13
Chapter 5 - Event Contracts, Schema Evolution, Retention, Compaction, and Tombstones	16
Chapter 6 - Outbox, Inbox, Retries, DLT, and Exactly-Once Boundaries	19
Chapter 7 - Spring Kafka: Producers, Listeners, Acknowledgements, Errors, Transactions, and Tests	22
Chapter 8 - Capacity, Observability, Security, and Incident Response	25
Chapter 9 - Kafka/Spring Kafka Live Interviews, Rapid Q&A, Practice, and Sources	28
Chapter 10 - Java 21 Kafka Delivery Reasoning Companion	32
Part II - Practice and Handoff	35

Part I - Core Learning

SDE-2 CORE CHAPTER

Chapter 1 - Apache Kafka and Spring Kafka

Learning Path: Event Log Before Listener Annotations

Kafka stores ordered records in partitioned logs. Producers append; consumers fetch by offset; consumer groups coordinate partition ownership. Spring Kafka automates clients and listener containers, but it cannot choose the correct key, commit point, retry effect, or schema contract for you.

From Vinay: In an interview, draw the partition and write the offset beside each record. Then mark where the business side effect happens and where the offset is committed. Most "exactly once" confusion disappears on that timeline.

Dependency order

TEXT

```
event contract and log
-> topic + partitions + replication
-> producer key/batch/ack/idempotence
-> consumer fetch/group/offset/rebalance
-> delivery and failure windows
-> schema evolution + retention/compaction
-> outbox/inbox/retry/DLT/transactions
-> Spring Kafka containers and handlers
-> capacity, security, and incidents
```

One running event

JSON

```
{
  "eventId": "evt-7f3",
  "eventType": "OrderPaid",
  "schemaVersion": 2,
  "occurredAt": "2026-08-02T10:00:00Z",
  "aggregateId": "order-42",
  "aggregateVersion": 4,
  "tenantId": "tenant-9",
  "data": { "amountCents": 5000, "currency": "USD" }
}
```

Use `order-42` as the record key when per-order order matters. Events for that key map to one partition under stable partitioning metadata. Ordering is not global across partitions.

End-to-end flow

TEXT

```
database transaction writes order + outbox row
-> publisher reads outbox
-> producer serializes/key-partitions/batches
-> broker leader appends and replicas copy
-> acknowledgement reaches producer
-> consumer group member fetches partition
-> deserializer validates schema
-> handler applies idempotent database effect
-> offset/checkpoint is committed
```

At every arrow ask: can it retry, duplicate, reorder, block, or lose acknowledgement?

Delivery terms

- **At-most-once:** commit/advance before processing; failure can lose work, duplicates reduced.
- **At-least-once:** process before commit; crash can repeat work, so effects must be idempotent.
- **Exactly-once Kafka processing:** transactions/idempotence can make supported Kafka reads/writes/offsets atomic for read-process-write flows. It does not automatically make MySQL, email, or HTTP side effects exactly once.

First failure timeline

TEXT

```
consume offset 25
charge database successfully
process crashes before committing offset 26
record 25 is fetched again
```

The correct default is not "avoid duplicate delivery." It is "make the charge/application effect detect the duplicate event ID or business command."

The SDE-2 answer frame

- 1 Event versus command: what fact/intent is represented?
- 2 Key and partition: what ordering and parallelism are required?
- 3 Producer: what acknowledgement, retry, idempotence, and timeout?
- 4 Consumer: when is the side effect durable and when is the next offset committed?
- 5 Poison/transient failure: blocking retry, retry topic, DLT, or stop?
- 6 Schema: how do old/new producers and consumers coexist?
- 7 Operations: retention, capacity, lag, rebalances, security, replay, recovery?

Practice

- **Foundation:** Draw two partitions and assign six keyed order events.
- **Interview Core:** Mark duplicate/loss windows around a database write and offset commit.
- **SDE-2 Follow-up:** Explain why a transactional Kafka producer cannot roll back a payment-provider call.

SDE-2 CORE CHAPTER

Chapter 2 - Broker Log, Partitions, Replication, and Storage Flow

Topic and partition

A topic is divided into partitions. Each partition is an ordered sequence of records with monotonically increasing offsets. The offset identifies a position in one partition, not a global event sequence and not a permanent business ID.

TEXT

```
orders-0: [offset 0 A] [1 C] [2 A]
orders-1: [offset 0 B] [1 D]
```

Consumers can process partitions in parallel. Within one group, a partition is assigned to at most one consumer at a time; extra consumers beyond the available partitions are idle for that topic assignment.

Key-to-partition behavior

A producer partitioner uses topic metadata and record key/value. Same key normally maps to the same partition while the partition count/partitioner contract remains compatible. Increasing partitions can change the key mapping for future records, so per-key ordering across the change needs a migration strategy.

Null keys may be distributed using batching/sticky behavior rather than deterministic entity ordering. Never require order and omit the key.

Replication

Each partition has a leader and replicas. Producers/consumers interact with leaders; followers fetch from leaders. An in-sync replica set tracks replicas sufficiently caught up under broker rules. Controller metadata/elections are coordinated by KRaft in current Kafka deployments.

TEXT

```
producer -> partition leader append
            |-> follower fetch/apply
            |-> follower fetch/apply
            high watermark advances
consumer fetches committed/available records per isolation
```

Replication factor, `acks`, `min.insync.replicas`, and unclean-election policy jointly affect durability and availability. `acks=all` means all required in-sync acknowledgements under current ISR/minimum settings-not every configured replica regardless of health.

Append and storage intuition

Partitions are stored as log segments with index structures. Records are appended, batched, compressed, served through sequential I/O/page cache, and removed/compacted by policy. Kafka does not keep every message as an individual file or deserialize application payloads to route them.

Large batches improve throughput/compression but increase memory and latency while filling. Large records stress broker/client buffers, replication, recovery, and consumers. Prefer object storage plus a referenced immutable blob for very large payloads when appropriate.

High watermark and log end

The leader's log end can include records not yet safely replicated/visible under a consumer's isolation. The high watermark bounds records considered replicated enough for normal consumption. Transactional aborted/uncommitted records add another visibility rule for `read_committed` consumers.

Retention and deletion

Time/size retention removes old log segments; it is not per-record exact deletion. Consumer lag beyond retained history can make offsets invalid and force reset/rebuild. Retention must exceed the longest supported outage/replay window plus safety margin, subject to storage capacity.

Capacity worksheet

Estimate:

TEXT

```
ingress bytes/sec = events/sec * average encoded bytes
replicated network ~= ingress * replication factor (rough planning input)
retained bytes ~= ingress * retention seconds * replication factor
consumer egress = ingress * independent consuming groups
```

Then account for compression, peaks, headers, indexes, compaction, replication catch-up, reassignments, and disk/headroom. Measure rather than treating the formula as exact.

Edge matrix

Trap	Consequence	Repair
global-order assumption	events interleave across partitions	key/one partition or downstream versioning
add partitions casually	future key mapping changes	migration/versioned routing
RF 3 means no loss	acknowledgement/ISR/election policy still matters	test failure contract
retention too short	lagging group cannot replay	size from outage/recovery objective
giant records	broker/client/recovery pressure	compact event or external blob

Practice

- **Foundation:** Explain why offset 10 on partition 0 is unrelated to offset 10 on partition 1.
- **Interview Core:** Choose partitions for 60 consumer instances and justify peak throughput/key skew.
- **SDE-2 Follow-up:** Plan a partition-count increase while preserving aggregate ordering.

SDE-2 CORE CHAPTER

Chapter 3 - Producers: Keys, Batching, Acknowledgements, Idempotence, and Transactions

Producer request flow

TEXT

```
send(record)
-> serialize key/value/headers
-> fetch/use metadata
-> partition selection
-> per-partition accumulator batch
-> sender transmits ProduceRequest
-> leader append + replication requirement
-> callback/future success or failure
```

`send` success normally means the configured broker acknowledgement was received, not that a consumer processed the event.

Keys and partitioning

Choose a key for the ordering/ownership boundary: order ID, account ID, or customer ID. A key with extreme skew creates a hot partition; randomizing it sacrifices ordered processing. Sometimes shard a high-volume aggregate by a subkey and carry sequence/version for downstream reconciliation.

Custom partitioners become a long-lived compatibility contract. Rolling producers with different partition logic can split one key's events.

Batching and compression

Producer accumulators batch per partition. `batch.size`, `linger.ms`, compression, buffer memory, and in-flight requests trade latency, throughput, CPU, and memory. A low-traffic partition may not fill a batch. Compression works better across similar records; broker stores/transfers batches efficiently.

Never tune only events/sec. Observe record size, compression ratio, request rate/size, buffer wait, errors/retries, and p99 acknowledgement latency.

Acknowledgements

- `acks=0`: producer does not wait for broker acknowledgement; loss visibility is weak.
- `acks=1`: leader acknowledges without waiting for all required replicas.
- `acks=all`: leader waits for the current required in-sync replicas; combine with suitable replication/min ISR.

Durability also depends on broker/topic configuration and failure mode. Stronger acknowledgement may reject writes during insufficient ISR-a deliberate availability trade-off.

Retries and idempotent producer

Without idempotence, a response can be lost after append and retry can duplicate. Idempotent producer protocol uses producer identity/epoch and per-partition sequence numbers to deduplicate supported retries and preserve ordering under configured constraints.

It does not deduplicate a new application request with a new producer/session, nor a consumer's database side effect. Keep event IDs/business idempotency.

Timeouts include delivery budget, request timeout, metadata/serialization/buffer wait. A future failure does not always mean the broker stored nothing; classify the exception and preserve the event ID/outbox state.

Kafka transactions

A transactional producer can atomically publish records across partitions/topics and commit consumed offsets as part of a consume-transform-produce transaction. Consumers using `read_committed` hide aborted transactional records.

TEXT

```
begin transaction
  send output records
  send consumer offsets to transaction
commit transaction
```

Stable unique `transactional.id` ownership enables fencing of old producer instances. Mismanaging identity across replicas can fence healthy producers or allow conflicts.

Kafka transactions do not include arbitrary database writes. For DB-to-Kafka use a transactional outbox; for Kafka-to-DB use idempotent inbox/business writes and commit offsets after durable effect.

Producer failure matrix

Failure	Outcome	Response
serialization error	no broker send	fix/quarantine invalid event
buffer exhausted	backpressure/timeout	bound producers and tune/load shed
insufficient ISR	write rejected under durability policy	wait/retry within delivery budget, alert
ack lost after append	result unknown	idempotent retry/outbox reconciliation
producer fenced	transaction identity conflict/old instance	stop and fix ownership
oversized record	broker/client rejection	shrink/externalize payload

Practice

- **Foundation:** Explain `acks=all` with replication factor 3 and min ISR 2.
- **Interview Core:** Select a key for account balance events and handle one hot account.
- **SDE-2 Follow-up:** Design DB commit plus publish using an outbox, stable event ID, idempotent producer, and publisher checkpoint.

SDE-2 CORE CHAPTER

Chapter 4 - Consumers: Groups, Offsets, Polling, Rebalances, and Delivery

Consumer group ownership

Consumers with the same group ID divide subscribed partitions. Different groups independently consume the same topic. A rebalance changes ownership when membership, subscriptions, or partitions change.

Generation/member protocols prevent stale members from committing as if they still owned a partition, but your external side effect may already have happened.

Poll loop

TEXT

```
poll -> receive records from assigned partitions
-> deserialize/validate
-> process in partition order
-> make side effect durable
-> commit next offset
-> poll again before liveness deadline
```

Committed offset is usually the **next** record to read. After processing offset 25, commit 26.

Commit timing windows

TEXT

```
commit 26 -> then process 25 -> crash = possible loss (at-most-once)
process 25 -> then commit 26 -> crash between = duplicate (at-least-once)
```

At-least-once plus idempotent effect is the common safe baseline.

Auto commit does not mean "commit after every successful handler." It commits positions according to polling/time behavior and can advance beyond safely processed asynchronous work. Understand framework/container semantics before enabling it.

Idempotent inbox

SQL

```
BEGIN;  
INSERT INTO processed_event(consumer_name, event_id)  
VALUES (?, ?); -- unique constraint  
-- apply business mutation in same database transaction  
COMMIT;
```

Duplicate event insert fails or reports prior processing; treat it as already applied. Commit Kafka offset afterward. A crash after DB commit but before offset commit repeats the event, and the inbox makes it harmless.

Polling and long processing

`max.poll.interval.ms` bounds time between polls before a consumer is considered failed for group progress. `max.poll.records` bounds each batch. Session timeout/heartbeats relate to membership liveness. Client versions may run heartbeats separately, but a blocked application still violates poll interval.

For slow work, reduce batch size, move work to bounded worker queues while pausing partitions carefully, extend interval with evidence, or redesign into stages. Unbounded async submission can commit ahead, reorder one partition, exhaust memory, and trigger rebalances.

Rebalances

During revocation, finish/commit only work known durable for revoked partitions, stop accepting new work, and release resources. Cooperative rebalancing can reduce stop-the-world movement but does not remove correctness requirements. Static membership can reduce churn for stable instances but increases stale-member recovery considerations.

Seeking and replay

Offsets can be reset to earliest/latest/timestamp/specific position. Replay requires:

- retained data still exists;
- schemas and code can read historical versions;
- side effects are idempotent or directed to a new projection;
- downstream capacity can handle catch-up;
- progress and stop criteria are observable.

Never reset a production group casually to "fix lag." It can replay millions of effects.

Poison records

Separate transient (database unavailable), permanent data/schema (invalid payload), and code/invariant bugs. Infinite retry blocks a partition forever. A DLT/quarantine preserves evidence but is not success; include original topic/partition/offset, key, headers, exception class, schema version, attempt history, and remediation owner-without exposing secrets.

Edge matrix

Trap	Result	Repair
commit before async tasks finish	data loss	track contiguous durable offsets
process same partition concurrently	reorder	serialize per partition/key
one poison record	partition stuck or data skipped	bounded classified retry + quarantine
slow handler	poll timeout/rebalance loop	bound work/pause/adjust design
DLT with no replay tool	permanent graveyard	ownership, repair, replay workflow

Practice

- **Foundation:** Draw commit-before and commit-after failure windows.
- **Interview Core:** Build an idempotent MySQL inbox transaction.
- **SDE-2 Follow-up:** Process a polled batch concurrently while committing only the highest contiguous successful offset per partition.

SDE-2 CORE CHAPTER

Chapter 5 - Event Contracts, Schema Evolution, Retention, Compaction, and Tombstones

Event versus command

- **Event:** immutable fact in past tense (`OrderPaid`). Consumers choose reactions.
- **Command:** request for an owner to act (`CapturePayment`). It may be rejected and often has one logical handler.

Avoid ambiguous "OrderUpdate" payloads. Include stable identity, type, occurrence time, aggregate/version, tenant, and domain data. Transport metadata (topic, partition, offset) is not a business event ID.

Schema formats

JSON is readable but needs external validation/compatibility discipline. Avro, Protobuf, and JSON Schema plus a registry can enforce typed evolution and schema IDs. The best format depends on ecosystem, performance, language, governance, and evolution needs-not interview fashion.

Compatibility

Backward compatibility asks whether new consumers can read old data; forward compatibility asks whether old consumers can read new data. Full/transitive modes tighten combinations across history.

Common safe changes under many schemas:

- add optional/defaulted field;
- stop writing a field only after all readers tolerate absence;
- introduce new event type rather than change meaning;
- preserve numeric range and enum unknown handling.

Renaming is remove-plus-add unless aliases/migration are supported. Changing semantic meaning while retaining a field name is the most dangerous "compatible" change because syntax checks pass.

Consumer robustness

Consumers should ignore unknown fields when the contract permits, preserve unknown enum behavior, validate required business invariants, and quarantine truly unreadable events. Do not silently default a missing currency or tenant if that changes money/security meaning.

Deletion retention

Delete retention removes old segments by time/size. It supports replay only within the retained window. Topic storage is not an archival strategy unless retention/capacity/recovery are designed for it.

Log compaction

Compaction retains at least the latest known value per key over time while preserving record order/offset structure for retained records. Cleanup is asynchronous; old versions can remain for a while. Compaction is not a real-time unique-key table lookup and does not eliminate consumer duplicate handling.

A keyed null value (tombstone) represents deletion for compacted state. Tombstones themselves have retention/cleanup timing. Consumers building state must interpret deletion explicitly.

Never use null key for compacted state that needs per-entity latest value; the key is the identity.

Snapshot plus changelog

A consumer rebuilding state can restore a snapshot and replay from a recorded offset, or consume a compacted changelog from the beginning. Define atomicity between snapshot contents and offset, schema compatibility, tombstones, and validation/checksum.

Sensitive data

Kafka retention and replication spread event data. Minimize PII/secrets, encrypt in transit/at rest, enforce ACLs, redact logs/DLTs, and plan deletion requirements. Encrypting a field complicates filtering/debugging/key rotation; avoid publishing unnecessary sensitive data first.

Contract edge matrix

Change	Hidden risk	Safer rollout
add required field	old records/producers lack it	optional/default then migrate
reuse enum value	semantic corruption	new value/type and explicit mapping
rename topic immediately	mixed deploy loses traffic	dual publish/bridge with reconciliation
compact with null keys	no per-entity collapse	stable business key
DLT raw payload	PII retention leak	access/retention/redaction policy

Practice and solutions

- **Foundation:** Add an optional delivery note compatibly.
- **Interview Core:** Evolve amount from `int` to larger range without breaking readers; introduce long-compatible schema/version and test every reader before values exceed old range.
- **SDE-2 Follow-up:** Design GDPR deletion for a compacted customer projection; minimize original event data, emit keyed tombstones, control backups/DLTs, and verify downstream deletion rather than promising immediate compaction.

SDE-2 CORE CHAPTER

Chapter 6 - Outbox, Inbox, Retries, DLT, and Exactly-Once Boundaries

Database-to-Kafka dual-write problem

TEXT

```
commit database -> crash before Kafka send = missing event
send Kafka -> crash/DB rollback           = event for nonexistent state
```

There is no safe ordering of two independent commits by hope alone.

Transactional outbox

Write domain state and an outbox row in one database transaction:

SQL

```
BEGIN;
UPDATE purchase_order SET status='PAID', version=version+1
WHERE order_id=? AND version=?;
INSERT INTO outbox(event_id, aggregate_id, aggregate_version,
                  event_type, payload, status)
VALUES (?, ?, ?, 'OrderPaid', ?, 'NEW');
COMMIT;
```

A publisher polls with locking/claiming or uses CDC, publishes with stable key/event ID, and marks progress. Crash after Kafka acknowledgement but before marking sent creates a duplicate publish; consumers still need idempotency.

Outbox retention, poison payloads, ordering per aggregate, multiple publishers, monitoring, and replay are production requirements-not optional cleanup.

Kafka-to-database inbox

Store (`consumer`, `eventId`) with the business update in one DB transaction. Duplicate delivery becomes a unique conflict/prior success. Commit Kafka offset only after that transaction commits.

If events can arrive out of order despite same-key design (topic migration, multiple sources, manual replay), compare aggregate version and decide buffer, ignore older, or reconcile from source.

Retry taxonomy

Failure	Example	Response
transient dependency	DB connection/lock timeout	bounded exponential backoff with jitter
throttling/overload	downstream 429	honor budget, pause/bulkhead
permanent payload	invalid schema/domain	quarantine/DLT
code bug	null pointer for valid event	stop/alert or bounded retry then DLT with ownership
unknown side-effect result	HTTP timeout after request	idempotency lookup/reconciliation

Blocking retry preserves partition order but stops later records. Non-blocking retry topics free the main partition but change timing/order and create extra topics/consumers. Choose from ordering requirement and failure duration.

DLT lifecycle

A dead-letter topic is a controlled failure queue. Include original metadata and safe diagnostics. Define:

- alerts and owner/SLA;
- retention and PII access;
- classification and repair tool;
- replay key/order/destination;
- deduplication and maximum replay attempts;
- success verification and audit trail.

Publishing to DLT can fail. Decide whether to stop, retry, or retain the original offset; never silently commit and lose evidence.

Kafka exactly-once processing

For Kafka read $\rightarrow$ transform $\rightarrow$ Kafka write, a transaction can atomically publish outputs and consumed offsets. Requirements include idempotent/transactional producer, stable transactional IDs, transactional listener/container integration, and downstream `read_committed` where aborted records must be hidden.

It does not make a database update atomic. Some frameworks coordinate transaction managers in an order, but without distributed atomic commit a crash window remains. Use outbox/inbox/compensation and explain the window.

Side-effect state machine

For a payment request:

TEXT

RECEIVED -> CLAIMED -> PROVIDER_UNKNOWN -> SUCCEEDED/FAILED

Persist provider idempotency key and response/reference. A timeout moves to unknown, then a reconciliation worker queries provider status. Retrying with a new key is forbidden.

Practice and solutions

- **Foundation:** Draw the two DB/Kafka dual-write failure orders.
- **Interview Core:** Implement outbox publisher idempotency and lag metric.
- **SDE-2 Follow-up:** Compare blocking retry and retry topic for account events. Preserve per-account order with blocking/pause or an ordered state machine; retry topics explicitly sacrifice immediate order and need version reconciliation.

SDE-2 CORE CHAPTER

Chapter 7 - Spring Kafka: Producers, Listeners, Acknowledgements, Errors, Transactions, and Tests

Spring Kafka wraps Kafka clients with `KafkaTemplate`, listener containers, conversion, error handlers, transactions, and test utilities. Start from the client contract, then configure the abstraction explicitly.

Producer boundary

JAVA

```
CompletableFuture<SendResult<String, OrderPaid>> result =
    kafkaTemplate.send("order-events", orderId, event);

result.whenComplete((sent, failure) -> {
    if (failure != null) {
        // record classified failure; outbox remains pending
    }
});
```

Do not fire-and-forget without observing the future. The callback may run on client/framework threads; do not block it with slow I/O. An outbox publisher marks sent only after success and still tolerates duplicate publish after a crash.

Listener boundary

JAVA

```
@KafkaListener(topics = "order-events", groupId = "billing-v2")
public void onOrderPaid(ConsumerRecord<String, OrderPaid> record) {
    billingService.applyIdempotently(
        record.value().eventId(),
        record.key(),
        record.value());
}
```

Container acknowledgement modes determine when offsets are committed relative to records/batches/manual acknowledgement. Version-specific names/semantics should be verified in the Spring Kafka 4.1 reference. Select a mode only after the database side effect and error behavior are defined.

Manual acknowledgement does not create exactly once; acknowledging before async work is durable loses records. With asynchronous processing, track contiguous completed offsets per partition and pause/resume safely.

Deserialization failures

A normal listener may never receive a record that fails deserialization. Configure error-handling deserializers or byte-level/quarantine paths so metadata and raw safe payload can be diagnosed. Bound payload logging and protect PII.

DefaultErrorHandler

Classify retryable and nonretryable exceptions, configure bounded backoff, and use `DeadLetterPublishingRecoverer` when appropriate. Validate whether recovered offsets are committed and what happens when DLT publication fails.

JAVA

```
DeadLetterPublishingRecoverer recoverer =  
    new DeadLetterPublishingRecoverer(template,  
        (record, exception) ->  
            new TopicPartition(record.topic() + ".DLT", record.partition()));  
  
DefaultErrorHandler handler =  
    new DefaultErrorHandler(recoverer, new FixedBackOff(1_000L, 2L));
```

Two retries means three total attempts in this example. Infinite retry is rarely acceptable.

Non-blocking retry topics

`@RetryableTopic`/retry-topic configuration forwards failed records through delayed retry topics. It improves main-topic progress for longer failures but loses the original topic's ordering relationship and, in current Spring Kafka documentation, does not combine with container transactions. Batch-listener support also differs. Label and test the selected version.

Transactions

Spring can configure transactional `KafkaTemplate` producers and transactional listener containers. For read-process-write Kafka flows, container transactions coordinate output sends and offsets. If the listener throws, transaction rollback and after-rollback processing determine seeks/recovery.

A `@Transactional` database method plus Kafka transaction manager is not automatically a distributed atomic transaction. State commit order and crash window, or use outbox/inbox.

Testing

- unit test event mapping/idempotency/error classification;
- `MockProducer`/`MockConsumer` for client decisions;
- embedded KRaft broker for real protocol/topic/group behavior;
- target cluster for security, quotas, multi-broker replication/failover;
- fault injection for rebalance, DLT failure, DB commit/offset crash.

Assert key/partition, headers/schema, event ID, offset commit point, duplicate result, retry count/backoff, DLT metadata, and lag recovery-not just "listener method called."

Spring edge matrix

Mistake	Failure
ignore send future	silent/late publish failure
DLT every exception	transient outages become data quarantine
async handler + early ack	data loss
retry topic for ordered workflow	later event overtakes earlier
DB + Kafka annotations assumed atomic	dual-commit crash window
raw payload logging	security/PII exposure

Practice

- **Foundation:** Explain the `KafkaTemplate` future and broker acknowledgement.
- **Interview Core:** Configure classified bounded retry plus DLT ownership.
- **SDE-2 Follow-up:** Write a test that crashes after DB commit but before offset commit and proves inbox deduplication.

SDE-2 CORE CHAPTER

Chapter 8 - Capacity, Observability, Security, and Incident Response

Capacity starts with partitions and bytes

Partitions provide parallelism but consume broker/controller/client resources. Too few constrain throughput; too many increase metadata, open files, leader-election/reassignment time, and operational cost. Size from measured per-partition throughput and consumer parallelism, then include growth/recovery headroom.

Test skew: average events/sec can hide one key driving one partition to saturation.

Producer signals

- record/send rate and encoded size;
- batch size, compression ratio, requests;
- buffer available/wait and queue time;
- acknowledgement latency p50/p95/p99;
- retries, errors by class, timeouts;
- metadata age and partition skew;
- transactional abort/fencing.

Consumer signals

- lag by group/topic/partition and lag age where derivable;
- processing rate/latency/error by event type;
- time between polls, batch size, pause duration;
- rebalances, assignments/revocations, commit failures;
- retry attempts/topic lag and DLT rate/age;
- idempotency duplicate count;
- oldest unprocessed business event time.

Lag is a symptom. Diagnose whether ingress rose, handler slowed, one partition is hot, dependencies fail, rebalances loop, deserialization blocks, or consumer capacity disappeared.

Broker/cluster signals

- under-replicated/offline partitions and ISR changes;
- produce/fetch request latency and queues;
- disk throughput/space, log directory health, page-cache pressure;
- network saturation and request size;
- controller/metadata health, leader imbalance;
- replica fetcher lag, reassignments;
- quota throttling and authentication/authorization failures.

Incident: lag grows on one partition

- 1 Identify hot partition/key and oldest event.
- 2 Compare its ingress/record size/handler latency with peers.
- 3 Check consumer assignment, dependency errors, retries, and poison record.
- 4 Protect downstream with backpressure; do not add consumers beyond partition count expecting relief.
- 5 Mitigate hot aggregate/key only if ordering can be preserved or downstream versions reconcile.
- 6 Reprocess/DLT with audit and add a regression/load test.

Incident: rebalance storm

Check deployment churn, poll interval violations, session/connectivity, GC pauses, overloaded coordinators, and unstable subscriptions. Reduce work per poll, pause safely, tune only with evidence, use cooperative/static membership where appropriate, and avoid synchronized rolling restarts.

Incident: disk filling

Check retention overrides, unexpected record size/rate, replication factor, compaction backlog, stuck consumers (which do not themselves stop deletion retention but may create business pressure), and failed log cleanup. Do not delete topic data manually. Adjust retention/capacity through controlled changes and preserve required replay/RPO.

Security

- TLS and authenticated SASL mechanism appropriate to environment;
- topic/group/transactional-ID ACLs with least privilege;
- quotas to control noisy producers/consumers;
- secrets rotation and client reconnect testing;
- schema registry and Connect security too;
- PII minimization, retention, DLT/log redaction;
- audit topic/config/ACL changes;
- network segmentation and broker listener separation.

Keys/headers can contain sensitive IDs and appear in logs/metrics. Hash/redact carefully while retaining diagnostic correlation.

Recovery drills

Test broker loss under required ISR, producer unknown outcomes, consumer restart after side effect, offset reset, DLT replay, schema rollback, outbox backlog, certificate rotation, partition reassignment, and restoration/rebuild from retained/archived data.

Failure matrix

Symptom	Bad reflex	Evidence-driven action
lag high	add consumers	check partitions/skew/dependency
producer timeout	retry forever	delivery budget, idempotence, ISR/latency
DLT rising	ignore because main flow moves	classify/own/repair/replay
under-replicated partitions	lower min ISR immediately	fix disk/network/broker capacity first
auth failures during rotation	roll back blindly	verify credential overlap/client refresh

Practice

- **Interview Core:** Define alerts that distinguish total lag from one hot partition.
- **SDE-2 Follow-up:** Create a runbook for a 10× lag spike during a database outage, including backpressure, retries, DLT policy, recovery rate, and correctness verification.

SDE-2 CORE CHAPTER

Chapter 9 - Kafka/Spring Kafka Live Interviews, Rapid Q&A, Practice, and Sources

Live interview 1: ordering

Interviewer: "Guarantee order status events are processed in order."

Candidate: "Key by order ID so one order maps to one partition, process that partition serially, and carry aggregate version to reject/reconcile stale events. Ordering is only per partition. I would control partition-count/partitioner migrations because they can remap keys."

Live interview 2: DB plus publish

Interviewer: "Update order and publish `OrderPaid` atomically."

Candidate: "Write order and outbox in one MySQL transaction. A poller/CDC publisher sends stable event ID/key and marks progress. Crash after publish can duplicate; consumers use an inbox/idempotent effect. A Kafka transaction alone cannot include the database commit."

Live interview 3: duplicate charge

Interviewer: "Consumer charged twice."

Candidate: "It likely crashed after provider/DB effect and before offset commit. Persist event ID/provider idempotency key and business update in one durable state machine/inbox. On unknown provider timeout reconcile with the same key. Offset commits after the durable decision."

Live interview 4: poison record

Interviewer: "One malformed record blocks a partition."

Candidate: "Classify deserialization versus transient dependency versus code bug. Capture raw bytes safely before normal conversion, use bounded attempts, publish metadata to a DLT/quarantine with an owner, then commit only after recovery publication succeeds. Provide repair/replay tooling and preserve key/order implications."

Live interview 5: lag spike

Interviewer: "Lag is one million; add consumers?"

Candidate: "First inspect lag per partition, ingress/processing rates, key skew, retries, rebalances, and downstream latency. More consumers help only up to partition count and not one hot partition. I would protect dependencies, estimate drain time, and verify idempotency during accelerated catch-up."

Live interview 6: exactly once

Interviewer: "Kafka says exactly once. Are payments exactly once?"

Candidate: "Kafka transactions can atomically commit Kafka outputs and input offsets for supported read-process-write flows with `read_committed`. The payment API is outside that transaction. Payments need provider idempotency, durable state, reconciliation, and inbox/outbox handling."

Live interview 7: Spring retry topics

Interviewer: "Use `@RetryableTopic` for account events?"

Candidate: "Non-blocking retry frees the main partition but a later account event can overtake the failed one; current Spring Kafka also does not combine it with container transactions. If account order is invariant, use blocking retry/pause with bounded outage handling or a versioned state machine that safely reconciles reorder."

Rapid answered questions

- 1 **Ordering scope?** One partition, not the topic globally.
- 2 **Offset meaning?** Position in one partition; commit normally stores the next record to read.
- 3 **More consumers than partitions?** Extra members are idle for that assignment.
- 4 **Why key?** Stable routing/order boundary; null keys do not preserve entity order.
- 5 **Does RF 3 guarantee no loss?** No; acknowledgement, ISR, election, durability, and failures matter.
- 6 **acks=all means every replica?** It waits for required current in-sync replicas under configuration, not every configured replica regardless of health.
- 7 **Idempotent producer scope?** Deduplicates supported producer retries; not arbitrary app requests/consumer effects.
- 8 **At-least-once window?** Effect succeeds, crash before offset commit, record repeats.
- 9 **At-most-once window?** Commit first, crash before effect, record lost to that group.
- 10 **What is rebalance?** Partition ownership changes across group members.
- 11 **Why max.poll.interval?** Bounds time between polls/progress before membership is reconsidered.
- 12 **Outbox purpose?** Atomically records domain change and publication intent in one DB.
- 13 **Inbox purpose?** Deduplicates a consumer effect in the same DB transaction.
- 14 **DLT equals success?** No; it is observable quarantine requiring repair/replay ownership.
- 15 **Compaction keeps only one record immediately?** No; cleanup is asynchronous and old versions can remain.
- 16 **Tombstone?** Keyed null value representing deletion in compacted state.
- 17 **Can partitions be increased safely?** Capacity yes, but key mapping/order compatibility must be planned.
- 18 **Spring manual ack equals exactly once?** No; commit/effect timeline still determines loss/duplicates.
- 19 **Retry topic preserves order?** Not relative to later main-topic records.
- 20 **Consumer lag root cause?** It is a symptom; examine rate, skew, dependencies, errors, and membership.

Cumulative assessment

Design an order-event platform: topic/partition/key/RF/retention, schema compatibility, outbox publisher, idempotent billing consumer, retry/DLT, Spring configuration, 10× load capacity, observability, security, replay, and broker/DB outage timelines.

Strong solution: quantifies bytes/partitions, preserves per-order versions, explains acknowledgement and offset windows, uses outbox/inbox, classifies failures, treats DLT as owned work, separates Kafka EOS from external effects, and validates multi-broker behavior.

Authoritative references

- [Apache Kafka documentation](#): design, producer, consumer groups, replication, transactions, configuration, security, and operations.
- [Spring for Apache Kafka 4.1 reference](#): templates, containers, acknowledgement, transactions, error handlers, retry topics, DLT, testing, and observability.
- Schema-format/registry documentation selected by the deployment.

The JVM-local lab validates client and Spring configuration behavior. A target multi-broker/KRaft environment remains required for replication, ISR, controller, failover, security, quotas, and reassignment claims.

SDE-2 CORE CHAPTER

Chapter 10 - Java 21 Kafka Delivery Reasoning Companion

Executable models for record identity, contiguous offset commits, idempotent inbox effects, failure classification, consumer parallelism, and aggregate ordering.

JAVA (continued 1/3)

```
import java.util.HashSet;
import java.util.Objects;
import java.util.Set;
import java.util.TreeSet;

/** Dependency-free executable reasoning models for Kafka and Spring Kafka. */
public final class KafkaInterviewCompanion {
    private KafkaInterviewCompanion() {}

    record RecordId(String topic, int partition, long offset) {
        RecordId {
            Objects.requireNonNull(topic, "topic");
            if (partition < 0 || offset < 0) {
                throw new IllegalArgumentException("negative partition or offset");
            }
        }
    }

    static final class ContiguousCommitTracker {
        private final TreeSet<Long> completed = new TreeSet<>();
        private long nextOffset;

        ContiguousCommitTracker(long firstOffset) {
            nextOffset = firstOffset;
        }

        long complete(long offset) {
            if (offset >= nextOffset) {
                completed.add(offset);
            }
            while (completed.remove(nextOffset)) {
                nextOffset++;
            }
            return nextOffset;
        }
    }
}
```

JAVA (continued 2/3)

```
    }  
}  
  
static final class Inbox {  
    private final Set<String> processedEventIds = new HashSet<>();  
    private long totalCents;  
  
    boolean apply(String eventId, long deltaCents) {  
        Objects.requireNonNull(eventId, "eventId");  
        if (!processedEventIds.add(eventId)) {  
            return false;  
        }  
        totalCents = Math.addExact(totalCents, deltaCents);  
        return true;  
    }  
}  
  
enum FailureKind { TRANSIENT, PERMANENT_DATA, CODE_BUG, UNKNOWN_SIDE_EFFECT }  
enum Handling { RETRY_WITH_BACKOFF, QUARANTINE, STOP_AND_ALERT, RECONCILE }  
  
static Handling handling(FailureKind failure) {  
    return switch (failure) {  
        case TRANSIENT -> Handling.RETRY_WITH_BACKOFF;  
        case PERMANENT_DATA -> Handling.QUARANTINE;  
        case CODE_BUG -> Handling.STOP_AND_ALERT;  
        case UNKNOWN_SIDE_EFFECT -> Handling.RECONCILE;  
    };  
}  
  
static int requiredConsumers(int partitions, int desiredConcurrency) {  
    if (partitions <= 0 || desiredConcurrency <= 0) {  
        throw new IllegalArgumentException("positive values required");  
    }  
    return Math.min(partitions, desiredConcurrency);  
}
```

JAVA (continued 3/3)

```
}

record AggregateEvent(String aggregateId, long aggregateVersion) {
    AggregateEvent {
        Objects.requireNonNull(aggregateId, "aggregateId");
    }

    boolean isNextAfter(long appliedVersion) {
        return aggregateVersion == appliedVersion + 1;
    }
}

public static void main(String[] args) {
    ContiguousCommitTracker tracker = new ContiguousCommitTracker(10);
    assert tracker.complete(11) == 10;
    assert tracker.complete(10) == 12;
    assert tracker.complete(13) == 12;
    assert tracker.complete(12) == 14;

    Inbox inbox = new Inbox();
    assert inbox.apply("event-1", 500);
    assert !inbox.apply("event-1", 500);
    assert inbox.totalCents == 500;

    assert handling(FailureKind.TRANSIENT) == Handling.RETRY_WITH_BACKOFF;
    assert handling(FailureKind.UNKNOWN_SIDE_EFFECT) == Handling.RECONCILE;
    assert requiredConsumers(6, 10) == 6;
    assert new AggregateEvent("order-42", 4).isNextAfter(3);

    RecordId record = new RecordId("orders", 2, 25);
    assert record.offset() + 1 == 26;
    System.out.println("KafkaInterviewCompanion checks passed");
}
}
```

Part II - Practice and Handoff

TRY IT | Practice Ladder and Next Steps

TRY IT | Practice ladder

- Foundation: log, partitions, producers, and consumers
- Interview Core: groups, offsets, delivery, and Spring integration
- SDE-2 Follow-up: retries, transactions, observability, and incidents

For every coding problem, state the input contract, select the numeric and collection types deliberately, write the invariant before the loop or recursion, test the smallest and largest valid inputs, and close with time and auxiliary-space complexity.

Navigation

[Previous book: FW 09 - Persistence, SQL, JPA, and Caching](#)

[Series index](#)

[Next book: FW 11 - Spring Ecosystem Extensions](#)

TRY IT | Completion check

- ☐ I can recognize the core patterns without being told the data structure or algorithm.
- ☐ I can implement the representative Java templates from memory.
- ☐ I can explain why each implementation is correct.
- ☐ I can test boundaries, invalid inputs, overflow, and adversarial shapes.
- ☐ I can answer at least one SDE-2 follow-up involving trade-offs or production constraints.

Series Roadmap

Choose Java, DSA, Frameworks, or System Design first, then follow the books inside that segment in order. The same series codes and positions appear on the website, PDF covers, and canonical download folders.

SEGMENT	BOOK	FOCUSED LEARNING PATH
Java	JAVA 01	Java Foundations for Problem Solving
Java	JAVA 02	Git and GitHub for Java Engineers
Java	JAVA 03	Maven and Gradle for Java Engineers
Java	JAVA 04	Language, OOP, and Modern Java
Java	JAVA 05	Collections, Streams, and I/O
Java	JAVA 06	JVM and Java Execution
Java	JAVA 07	Java Concurrency and the Memory Model
Java	JAVA 08	Performance, Diagnostics, and GC Incidents
Java	JAVA 09	Java Question Bank, Study Plan, and Reference
DSA	DSA 01	Time and Space Complexity for Java Interviews
DSA	DSA 02	Number Systems and Math Foundations for DSA Interviews
DSA	DSA 03	Number Systems Interview Patterns and Rapid Revision
DSA	DSA 04	Bit Manipulation in Java
DSA	DSA 05	Loop Mastery, Patterns, and Index Calculations
DSA	DSA 06	Arrays and Array Problem-Solving Patterns
DSA	DSA 07	Strings and String Problem-Solving Patterns
DSA	DSA 08	Hashing: Maps, Sets, Frequency, and Prefix State
DSA	DSA 09	Recursion and Backtracking in Java
DSA	DSA 10	Linked Lists: Pointer Reasoning and Mutation
DSA	DSA 11	Stacks, Queues, Deques, and Monotonic Patterns
DSA	DSA 12	Binary Search: Bounds, Answers, and Invariants
DSA	DSA 13	Trees, Binary Search Trees, and Tries
DSA	DSA 14	Heaps, Priority Queues, Selection, and Top-K
DSA	DSA 15	Graphs: Traversal, Ordering, Paths, and Union-Find
DSA	DSA 16	Greedy Algorithms: Recognition, Proofs, and Scheduling
DSA	DSA 17	Dynamic Programming: State, Transitions, and Optimization
Frameworks	FW 01	MySQL for Java Backend Interviews
Frameworks	FW 02	Hibernate and JPA for Java Backend Interviews
Frameworks	FW 03	Spring Framework for Java Engineers
Frameworks	FW 04	Spring Boot for Java Backend Engineers
Frameworks	FW 05	Spring Boot and REST APIs

SEGMENT	BOOK	FOCUSED LEARNING PATH
Frameworks	FW 06	Spring Data for Java Backend Engineers
Frameworks	FW 07	MongoDB for Java Backend Interviews
Frameworks	FW 08	Redis for Java Backend Interviews
Frameworks	FW 09	Persistence, SQL, JPA, and Caching
Frameworks	FW 10	Apache Kafka and Spring Kafka
Frameworks	FW 11	Spring Ecosystem Extensions
Frameworks	FW 12	Spring AI for Java Engineers
System Design	SD 01	Design, Backend, Testing, and Security
System Design	SD 02	Distributed Systems and System Design

All 40 publication editions are available now. Choose Java, DSA, Frameworks, or System Design first, then follow the books inside that shelf in order. Each deep-dive book remains independently printable and available on the web.

About the Author

Vinay Reddy Kalluri

Vinay Reddy Kalluri is a senior Java backend engineer and the author and editor of the Java SDE-2 Interview Preparation Series. His work spans high-throughput microservices, event-driven architecture, resilient APIs, large-scale data processing, cloud delivery, and production performance across healthcare and enterprise systems.

Editorial approach

Vinay sets the learning sequence and publication standard and performs the final review for Java accuracy, evidence, attribution, and release readiness. Individual contributors retain visible credit for accepted original work through AUTHORS.md and the public Git history.

What distinguishes his approach is the combination of hands-on system design and operational ownership. He treats timeouts, retries, idempotency, dead-letter recovery, data consistency, SQL behavior, caching, and observability as part of the design rather than afterthoughts. He has also mentored engineers and led modernization, migration, and reliability work across distributed services.

Selected engineering and academic highlights

- More than eight years building Java and Spring Boot services for high-throughput healthcare and enterprise platforms.
- Designed Kafka-based event systems that have processed more than one million events per hour, with explicit idempotency, retry, recovery, and observability controls.
- M.S. in Computer Science from the University of Missouri-Kansas City (3.9/4.0) and M.S. in Software Engineering from VIT University (9.3/10), where he was a Gold Medalist.
- Independent builder of Work Visa Insights, an immigration-data intelligence platform, and Play Together, a published Android application.

Why this series exists

Vinay created this series to turn fragmented interview preparation into a deliberate progression: learn the fundamentals, run the examples, predict behavior, debug failures, explain trade-offs, and only then move into advanced engineering. The books reflect the same principle he applies to production systems: clarity before cleverness, explicit contracts, measurable behavior, and reliable foundations.

Connect: [LinkedIn](#) | [GitHub](#)

Copyright and Publishing Notes

Copyright 2026 Vinay Reddy Kalluri and credited contributors. Open educational edition.

Book prose, exercises, diagrams, and PDFs are licensed under Creative Commons Attribution 4.0 International (CC BY 4.0). Build scripts and source code are licensed under MIT. Individual credit is recorded in AUTHORS.md and Git history.

Repository: <https://github.com/vinayreddykalluri/SDE2-Interview-Handbook>. Attribution does not imply endorsement. Java and related marks are owned by their respective holders.

Examples target Java 21 unless a section states otherwise. Validate release-specific APIs, JVM behavior, security, performance, licensing, and operational requirements before production use.

Series position: Frameworks, Data, and Messaging - Book 10 of 12 - Study Step FW-10. The local table of contents and PDF bookmarks navigate within this file; the roadmap and printed filenames navigate across the complete series.

Source provenance: curated from the independent Java SDE-2 master guide plus focused original material. Original master chapter numbers are labeled explicitly when retained in cross-references.